



ESTACIÓN 7 · COHORTE 2

Agentes de código

Orquestación de trabajo, review y memoria

Convertir intención en trabajo confiable

7

piezas del ciclo de
implementación

Tomar artefactos reales de AI-DLC.

Usar OpenSymphony como ejemplo.

Ejecutar con arnés.

Capturar memoria y sincronizar docs.

Diseñar una cola de trabajo orquestable.

Publicar tareas en Linear.

Revisar PRs automáticamente.

Coordinar trabajo requiere más que un buen arnés

Intención

Contexto

Scope

Dependencias

Ejecución

Validación

Review

Memoria

Idea: el sistema de trabajo decide qué se hace visible, qué se valida y qué aprende el equipo después de cada cambio.

Varias capas sostienen la ejecución



El arnés implementa. La orquestación conserva intención, prioridad, estado y continuidad.

El ejemplo viene de EntreVista AI

Estado

`aidlc-state.md` y `audit.md`.

Inception

Requirements, user stories, application design y unit of work.

Construction

Unit 6: `auth-lambda`.

AI-DLC

como fuente de intención para una cola ejecutable

`auth-lambda` es la primera unidad

Repo objetivo: `entrevista-auth`.

Runtime: Python 3.12 / FastAPI + Mangum.

Story principal: US-18, Authenticate Into Dashboard.

Wave 1 en la secuencia de construcción.

Servicio fundacional para dashboard.

Demo: usar una unidad real como entrada para planear trabajo publicable.

ORQUESTACIÓN

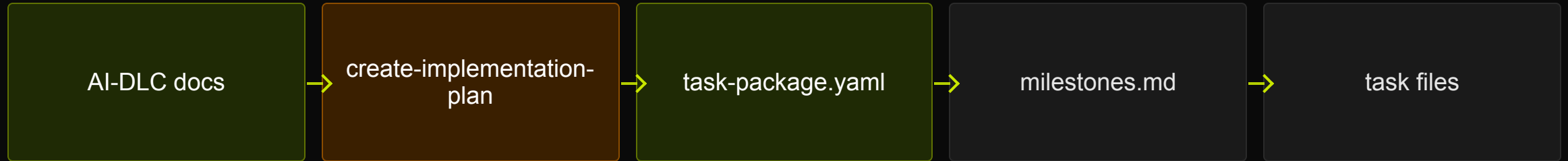
OpenSymphony materializa la cola de trabajo

Planning wave, task package, Linear, arnés, review y memoria.

La intención queda en un paquete revisable

<code>task-package.yaml</code>	<code>milestones.md</code>	Task files
IDs estables	Dependencias	Parent/sub-issues
Acceptance criteria	Test plan	Contexto

Primero se descompone el trabajo



Sigue demo: artefactos AI-DLC a paquete revisable.

CHECKPOINT

La cola se revisa antes de publicarse

Manifest completo.

Task files existentes.

Acceptance criteria medibles.

Contexto AI-DLC referenciado.

Milestones coherentes.

Dependencias válidas.

Test plans ejecutables.

Luego se proyecta al sistema de trabajo



Sigue demo: validate, dry-run, Linear.

Las tareas quedan listas para implementación

Issues en estado Todo.

Milestones por wave.

Parent/sub-issues.

Blocker relations.

Links al proyecto.

Overview actualizado.

Mapping local.

Una tarea publicada entra al arnés

Leer issue y task file.

Pedir plan corto.

Revisar scope y validación.

Aprobar implementación.

Ejecutar tests.

Adjuntar evidencia.

La autonomía funciona mejor cuando el trabajo llega con contexto, límites y una definición clara de evidencia.

CALIDAD

Review temprano reduce bugs y deuda técnica

Un segundo lector técnico mejora el ciclo antes del merge.

El review automatizado necesita evidencia

Intención

Cambio esperado y criterio de éxito.

Diff

Archivos, patrón de cambio y superficie tocada.

Tests

Comandos, resultados y huecos.

Logs

Screenshots o trazas relevantes.

Riesgos

Fallos probables y regresiones.

Áreas sensibles

Auth, datos, permisos y efectos externos.

Invariants

Reglas del codebase que sobreviven al cambio.

Reviewer humano

Notas para decidir con velocidad.

EJEMPLO

OpenHands PR Review implementa esa capa

Rol advisory.

Merge gate humano.

Same-repo PRs por defecto.

Evidence section requerida.

Comentarios high-signal.

Label de rerun: `review-this`.

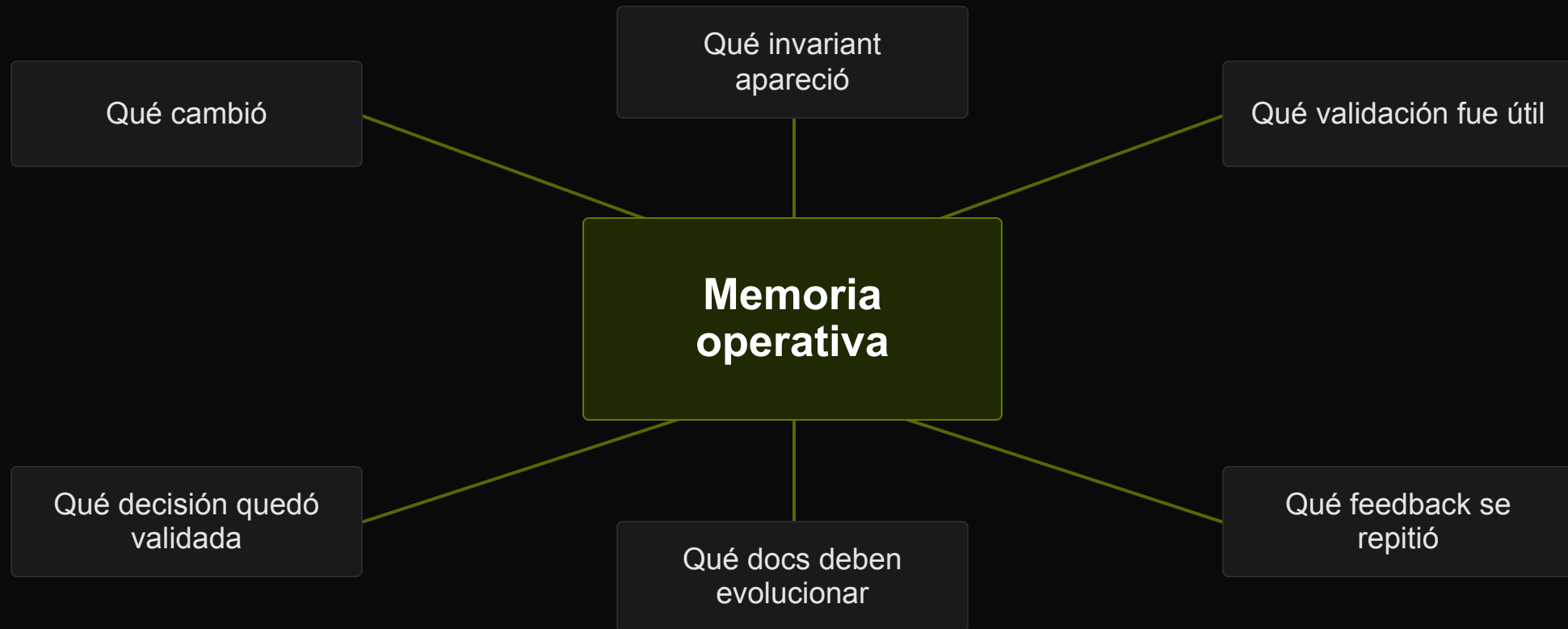
Setup explícito de secrets y variables.

CONTINUIDAD

El codebase debe volverse más fácil de entender

Cada tarea deja conocimiento para la siguiente.

La memoria captura conocimiento operativo



EJEMPLO

opensymphony memory implementa captura y consulta

Issue capsules.

DuckDB index.

`memory.yaml` .

Related issues.

Context bundle.

Topic docs.

Docs sync.

Lint de memoria y docs.

La memoria asiste antes de tocar código

```
opensymphony memory context --issue COE-456  
opensymphony memory related --paths crates/opensymphony-openhands  
opensymphony memory search "reconnect recovery"  
opensymphony memory docs --area authentication
```

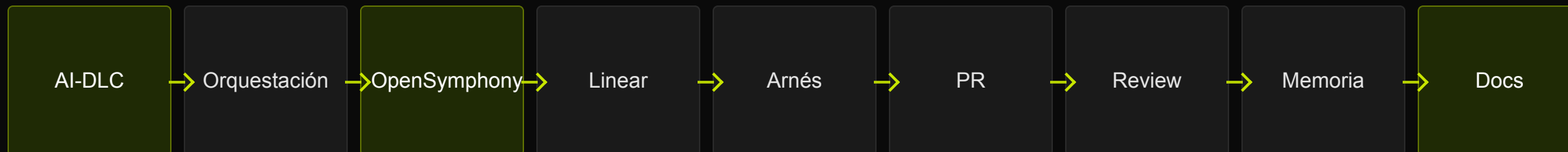
Uso conceptual: recuperar contexto local, decisiones previas y zonas del repo que pueden cambiar el plan.

La documentación recibe conocimiento estable

```
opensymphony memory capture COE-123 --dry-run  
opensymphony memory sync-docs --since-last-sync --dry-run  
opensymphony memory lint --public-docs
```

Key point: La memoria alimenta docs que ayudan al siguiente agente y al siguiente humano.

La estación completa el ciclo de implementación



Cada paso deja un artefacto verificable para que el siguiente paso arranque con menos fricción.

Publica una wave y cierra el ciclo

- ☐ `task-package.yaml`.
- ☐ `milestones.md`.
- ☐ Task files.
- ☐ Evidencia de validación.
- ☐ Issues en Linear Todo.
- ☐ PR con Evidence.
- ☐ AI PR review o setup.
- ☐ Memory capture dry-run.
- ☐ Docs sync dry-run.

La entrega buena hace visible el trabajo, la validación y el aprendizaje que queda disponible para la siguiente tarea.

Trabajo reusable, evidencia visible, memoria viva

AI-DLC preserva intención.

OpenSymphony estructura ejecución.

Linear publica coordinación.

El arnés implementa.

PR review reduce riesgo.

Memory convierte aprendizaje en contexto.

Cada merge deja un sistema que recuerda por qué el cambio existe y cómo se verificó.